

Track2Data User Guide

Turn [idtracker.ai](#) output into clean, calibrated, analysis-ready tables.

© 2026 Martina Bellio. Track2Data is released under the [MIT licence](#).

Contents

- [Introduction](#): what Track2Data does, how to install it, the quick start
- [Part I: The workflow, screen by screen](#)
 1. [Project](#)
 2. [Sessions](#)
 3. [Calibration](#)
 4. [Zones \(optional\)](#)
 5. [Metadata \(optional\)](#)
 6. [Preprocessing](#)
 7. [Metrics](#)
 8. [Processing](#)
 9. [Preview](#)
 10. [Export](#)
- [Part II: Reference](#)
 - [Understanding the output files](#)
 - [Troubleshooting](#)

Introduction

Track2Data turns [idtracker.ai](#) output folders into clean, calibrated, analysis-ready tables. This guide follows the desktop app one screen at a time, using a small demo project (two animals in a circular arena). Each screenshot sits next to the explanation of the screen it shows.

The screenshots are generated from the real app by `scripts/generate_guide_screenshots.py`, so they match the current version. In the app, **Help ► Open user guide** brings you here.

Install and start

```
pip install "track2data[ui]" # or, from a clone: pip install -e ".[ui]"
track2data-gui
```

Only need the command line? `pip install track2data` gives you `track2data run project.t2d.json` (see `track2data --help`).

How the workflow is organised

The left sidebar lists the stages. A badge shows where each one stands:

Badge	Meaning
✓	complete
△	usable, but look at the tooltip (e.g. a session has no stable identities)
✗	must be fixed before you can continue
○	nothing entered yet (optional stages stay ○)

Settings save themselves. There are no Apply buttons: a change is stored a moment after you make it, and again when you leave the screen. **Next** (bottom right) stays disabled, with a tooltip saying why, until Project, Sessions and Metrics are filled in and the calibration is valid.

#	Screen	What you do
1	Project	Create or open a project
2	Sessions	Add idtracker.ai output folders
3	Calibration	Choose how pixels become body lengths or cm
4	Zones	Draw regions of interest (optional)
5	Metadata	Attach treatment, date, ... from a CSV (optional)
6	Preprocessing	Gap filling, jump removal, smoothing
7	Metrics	Pick what to compute
8	Processing	Run the pipeline
9	Preview	Check trajectories, diagnostics and metric tables
10	Export	Write CSV / Feather / Excel and load them in R or Python

Quick start (five minutes)

1. **File ► New project...**, name it. You land on *Sessions*.
2. **Add Folders...** and choose one or more idtracker.ai session folders.
3. Leave *Calibration* on **Body length** unless you know your pixels-per-cm.
4. On *Metrics*, pick a preset such as **Standard locomotor**.
5. ► **Run pipeline**, then look at **Preview ► Trajectories** to confirm tracking looks right.
6. On *Export*, tick the formats you want and **Export**.

Do step 5 before trusting any number: the trajectory view is the quickest way to spot identity swaps, tracking gaps and over-aggressive smoothing.

Part I: The workflow, screen by screen

1. Project

A project is one `.t2d.json` file next to a folder of results. It stores every setting on the following screens, so you can close the app and pick up later, or re-run the identical analysis from the command line.

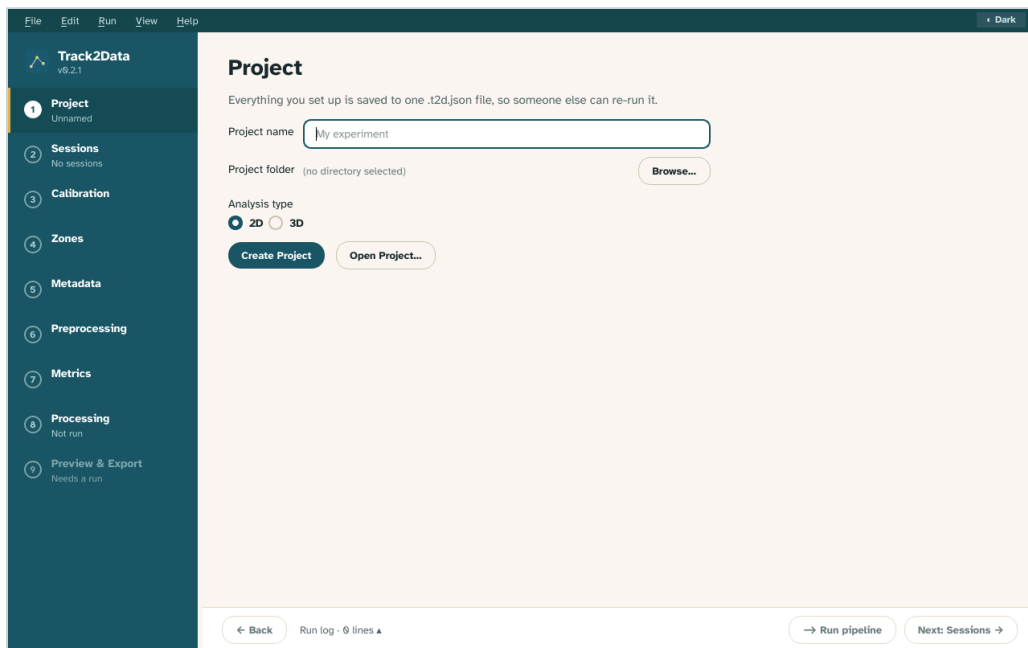


Figure 1. The Project screen, where every analysis starts.

Do this

1. **File ► New project...**, enter a name, choose a folder. Or **File ► Open project...** for an existing `.t2d.json`.
2. Track2Data moves straight on to [Sessions](#).

Notes

- **Changes are saved automatically** about half a second after you stop editing. The footer says **Unsaved changes** while one is pending and **Saved** once the file is on disk; if writing fails it says **Not saved** (hover for the reason) and your edits stay open. **File ► Save project** saves at once, including any value you have typed but not yet confirmed. The file is replaced atomically, so a failed save never leaves a half-written project.
- Closing the window, **New project** and **Open project** save first, and do nothing if saving fails. A reopened project saves back to the file it came from, and **New project** will not overwrite an existing project of the same name.
- Reopening a project re-reads each session folder in the background, so frame counts and calibration readiness appear again after a moment.

- Preprocessed sessions are cached in `.t2d_cache/` inside the project folder. It is safe to delete (`track2data cache clear --cache-dir <dir>` does it too); it is rebuilt when needed.
- **2D or 3D.** The **Analysis type** choice sets whether the project is a 2-D analysis (the default) or a 3-D one. 3D also asks how the recording was made: **One video, two panels** (top and side views in one frame) or **Two videos** (top and side tracked separately). The choice locks after the first session is added; remove all sessions to change it. A 3-D project can be set up and saved, but it cannot be run yet: Processing, Preview and Export say "3-D fusion is not available yet", and Calibration, Zones and Metrics apply to the 2-D tracks only.

2. Sessions

Add the folders your tracking software wrote its output to, one per recorded video. Adding is two steps: Track2Data looks inside what you picked, then asks you to confirm before anything joins the project.

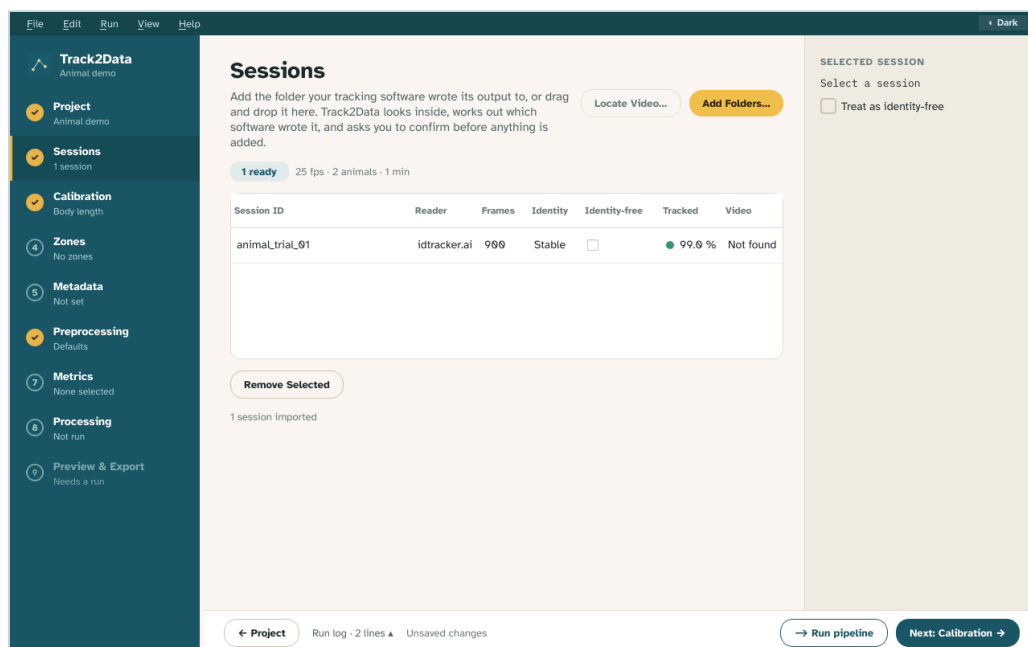


Figure 2. The Sessions screen lists every session with what was read from it.

Add sessions: scan, confirm, add

1. Press **Add Folders...**, or drag folders (or single tracking files) onto the screen. A progress bar with **Cancel** shows while Track2Data looks inside. Picking something else meanwhile replaces the scan.
2. When the scan ends, **Confirm tracking software** opens (Figure 3). It says which software wrote the folder, how sure it is (*High, Medium or Low confidence*) and why. If it chose wrongly, change **Software**.
3. Fill in any options the files do not record (a frame rate, a frame size, which keypoint stands for the animal). A required option shows *required* until you set it; Track2Data never guesses one, because a made-up frame rate would corrupt every speed.
4. Untick sessions you do not want, or rename them in the *Session* column. **Add** (it shows how many) stays disabled until adding would work, and its tooltip says what is missing.

Confirm tracking software

Scanned 20 entries in 0.0 s: /tmp/t2d_guide_8r799dd4/sessions/animal_trial_01

Software: idtracker.ai (High) ● High confidence

Why: 1 folder(s) with a trajectories/ subfolder holding trajectories.npy, trajectories_csv

	Session	Source	Note
☒	animal_trial_01	/tmp/t2d_guide_8r799dd4/sessions/animal_trial_01	

Cancel
Add 1 session

Figure 3. The confirm step: the software found, why, and the sessions about to be added.

A scan that finds nothing still opens the dialog and lists what it saw and what this version can read. A scan that fails writes its reason under the buttons. The chosen software is saved with each session, so reopening the project reads it the same way again.

Each row of the table shows what was read: the software (*Reader*), frame rate, number of frames, number of animals, **Identity** (Stable or Unstable) and **Video**.

Video: Found, Not found, Located

idtracker.ai records the path the video had on the computer it was tracked on, which is often not valid on yours. **Found** means the file is there. **Not found** means it is not; metrics are unaffected, but the video preview is unavailable. Select the row and press **Locate Video...** to point at the file; the row then reads **Located**, and the choice is stored in the project.

Identity-free

Tick this if the video was tracked *without* identification, or if identities swapped so often that "animal 1" is not one animal. Sessions flagged this way (by you or by idtracker.ai's own `track_wo_identities`) skip every metric that follows an individual across frames, because those numbers would be meaningless. Group metrics and the pooled zone-occupancy metrics still run. The sidebar shows ⚠ while any session is identity-free.

Good to know

- Folders are only read, never modified.
- Adding the same folder twice is ignored.
- The sidebar shows the session count under **Sessions**, and the chips above the table split them into ready and identity-free.
- Supported: idtracker.ai 6.x output (the legacy v5 layout is also read). v4 is not supported yet (Track2Data tells you so if it recognises a v4 folder; see [sending a v4 sample](#)).

3. Calibration

Calibration decides which units distances and speeds are reported in. Figure 4 shows the default, **Body length** mode.

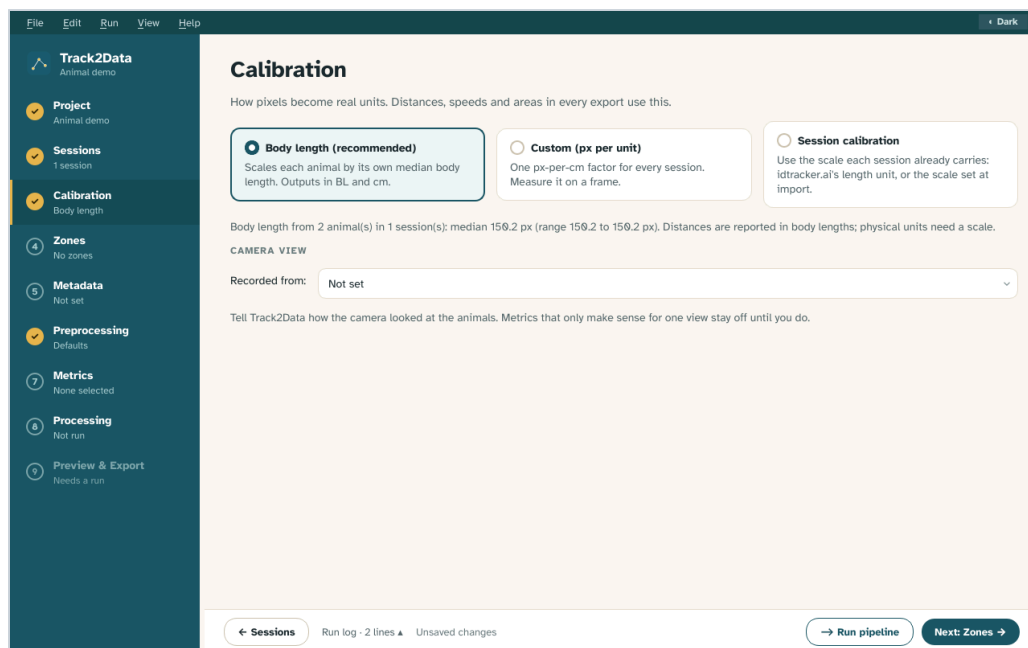


Figure 4. Calibration in Body length mode, with a summary of the body lengths read from your sessions.

Mode	Use it when	Reports
Body length (recommended)	You do not have a reliable scale	*_bl columns: distances in body lengths. Physical *_cm columns stay empty.
Custom (px per unit)	You know the scale	*_cm columns, and *_bl columns too whenever the tracker recorded body length
Session calibration	Each session already carries a scale: idtracker.ai's <i>Length Calibration</i> tool, a Ctrax / JAABA <code>trx.mat pxpermm</code> , or a scale you gave when importing	each session's own ratio; you must confirm the unit

In Session calibration mode, Track2Data also records how much your length-calibration clicks disagreed (a relative spread, saved in `sessions.csv` as `length_calibration_rel_sd`) and warns when it is above 5%. A warning means the scale is less certain, and so are the `*_cm` values.

In Body length mode the screen summarises the body lengths read from your sessions (median and range, in pixels) so you can sanity-check them.

Custom scale: measure on the frame

Pick **Custom**, then **Measure on frame...**. Click both ends of something of known length (a ruler, the arena diameter), type its real length, and press OK. The pixels-per-unit value is filled in for you. A third click starts the measurement over.

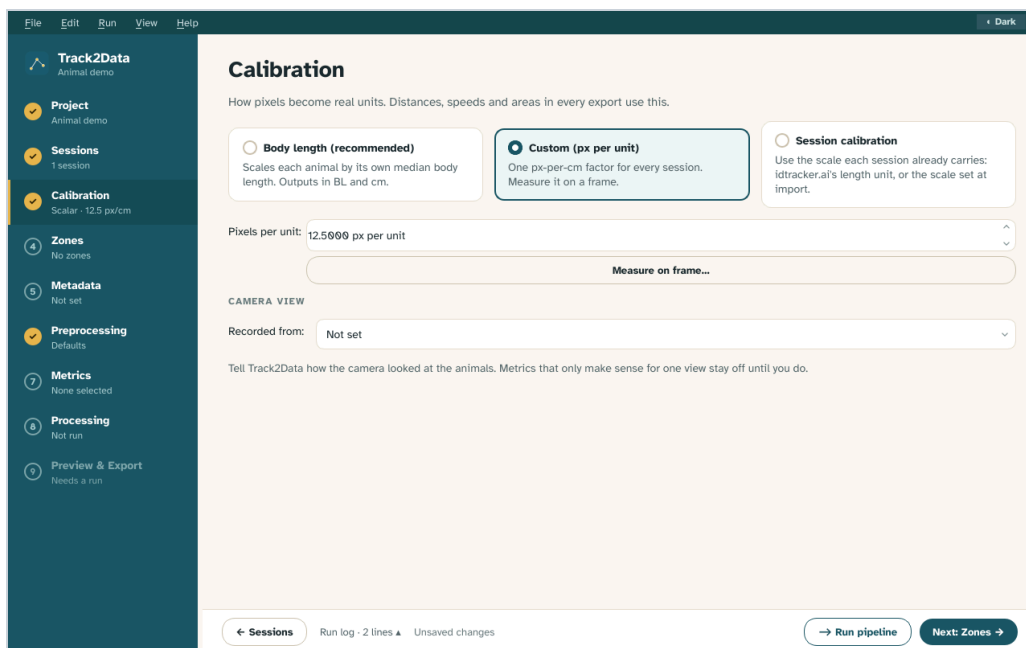


Figure 5. Custom calibration: click both ends of an object of known length to fill in the scale.

If a mode is incomplete (no scale, or an unconfirmed unit) the stage shows **X** and **Next** is disabled until you fix it.

Session calibration: where each tracker's scale comes from

Only some trackers record a scale. The ⓘ button beside the description opens a short note on what each tracker in your project records, with links to its documentation. Figure 6 shows the mode.

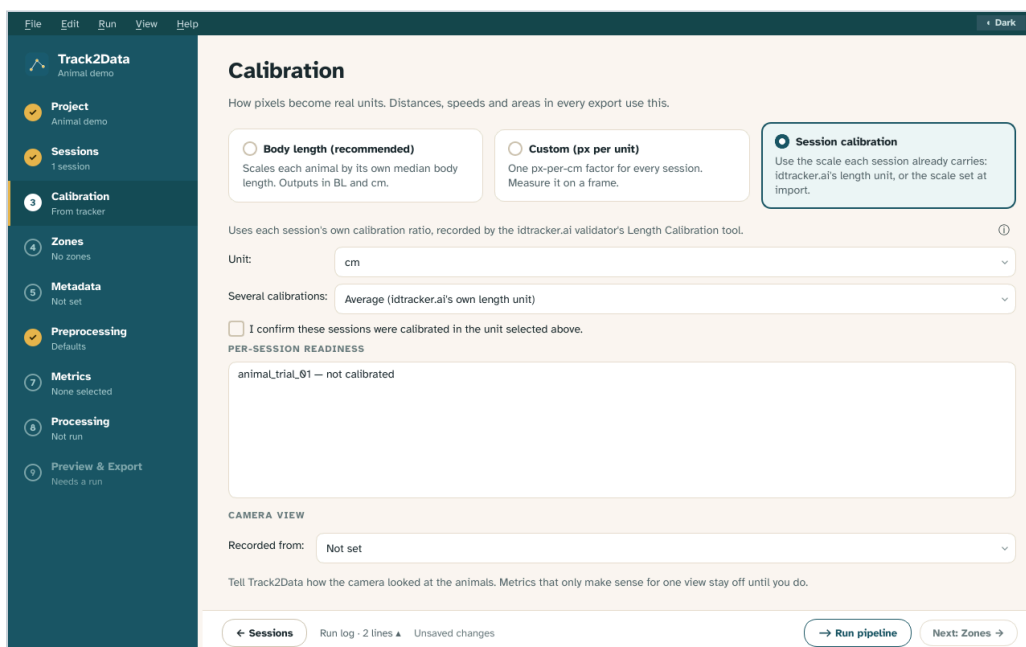


Figure 6. Session calibration: the unit, the choice between several calibrations, the confirmation and each session's readiness.

Tracker	Where the scale comes from
idtracker.ai	The Validator's <i>Length Calibration</i> tool saves one or more measurements. <code>length_unit</code> is their average: a factor from pixels to <i>user-defined units</i> . idtracker.ai does not record whether you measured in cm or mm, so the Unit you pick is your own declaration and you must confirm it. See the idtrackera.Session reference.
Ctrax / JAABA <code>trx.mat</code>	<code>pxpermm</code> (pixels per millimetre), present once the file's units were converted. Track2Data turns it into pixels per cm and ignores a value of exactly 1, which those tools use for "never calibrated". See the Ctrax trx fields .
DeepLabCut, SLEAP, Ctrax raw <code>.mat</code>	Their files record no scale. Give Scale (pixels per cm) in the confirm step when you add the sessions, or use Custom or Body length .

The per-session readiness list says whether each session is calibrated and with which value. A session without a scale reads *not calibrated* (idtracker.ai) or *no scale set at import* (other trackers). The **Session calibration** card is disabled when no session in the project can supply a scale.

Several calibrations. An idtracker.ai session can hold several length measurements. With **Several calibrations** you choose **Average** (idtracker.ai's own `length_unit`, the default) or **Median** (one badly placed click cannot move it). The list shows which one is in use, for example *median of 3*. If the saved `length_unit` differs from the average of the session's measurements (they were edited after export), the saved value is used and the run log carries a warning.

Camera view

Further down the same screen, **Camera view** says how the camera looked at the animals: **Not set** (the default), **Top-down**, or **Side view**. It is saved with the project, separately from the calibration, and a project that never sets it behaves exactly as before.

Some metrics only make sense for one view. Today that is **Vertical Position (Depth)**, IL-15, which needs a side view: on a top-down recording the image's vertical axis is not depth. Until you declare a side view, that row is greyed on the Metrics screen and its tooltip says how to switch it on; if it is selected anyway, the run skips it and the run's README says so. Declaring a view does not switch off any existing metric. Thigmotaxis (IL-14) and distance from the centre (IL-3) still assume you are looking down on the arena, so read them accordingly on a side view.

For depth, draw a **main** zone from the waterline to the floor on the Zones screen. The top edge is the surface and the bottom edge the floor; Track2Data never substitutes the video frame, so with no main zone the depth columns are empty and say why. For time in the upper or lower part of the tank, draw stacked secondary zones and use the zone metrics (Z-1 to Z-9).

4. Zones (optional)

Zones are regions of interest (the whole arena, a centre area, a feeder). Zone metrics need at least one. You can:

- **Import ROIs from Session:** use the polygons drawn in idtracker.ai's validator.
- **Load zones from CSV....**
- Draw your own, as described below.

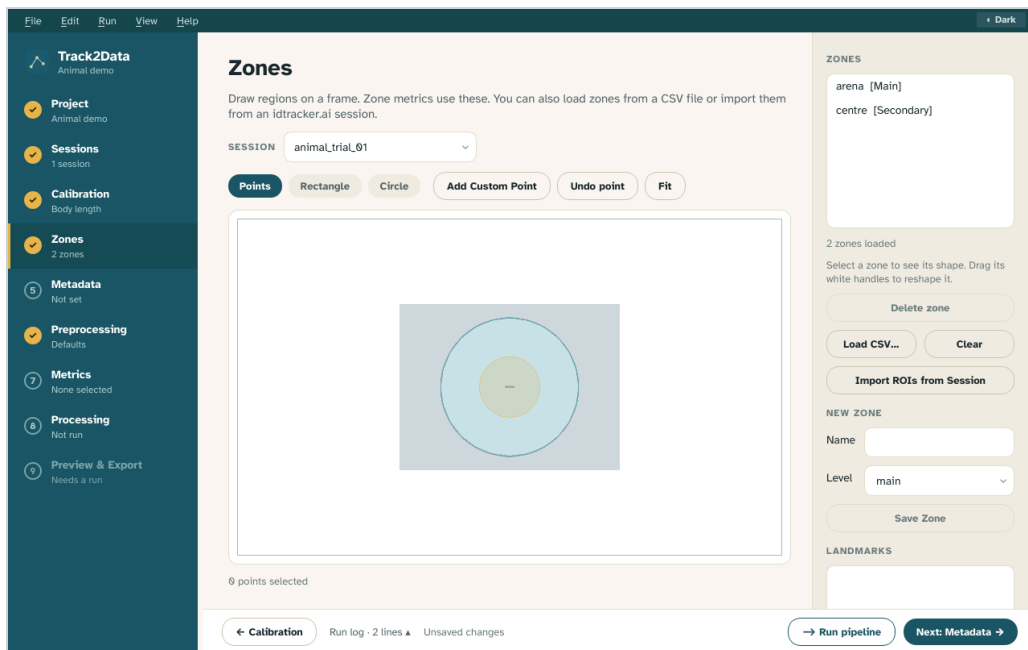


Figure 7. The zone list, with the import and load buttons.

Drawing zones

Scroll down to the canvas, which shows the session's background image. Saved zones appear shaded with their names.

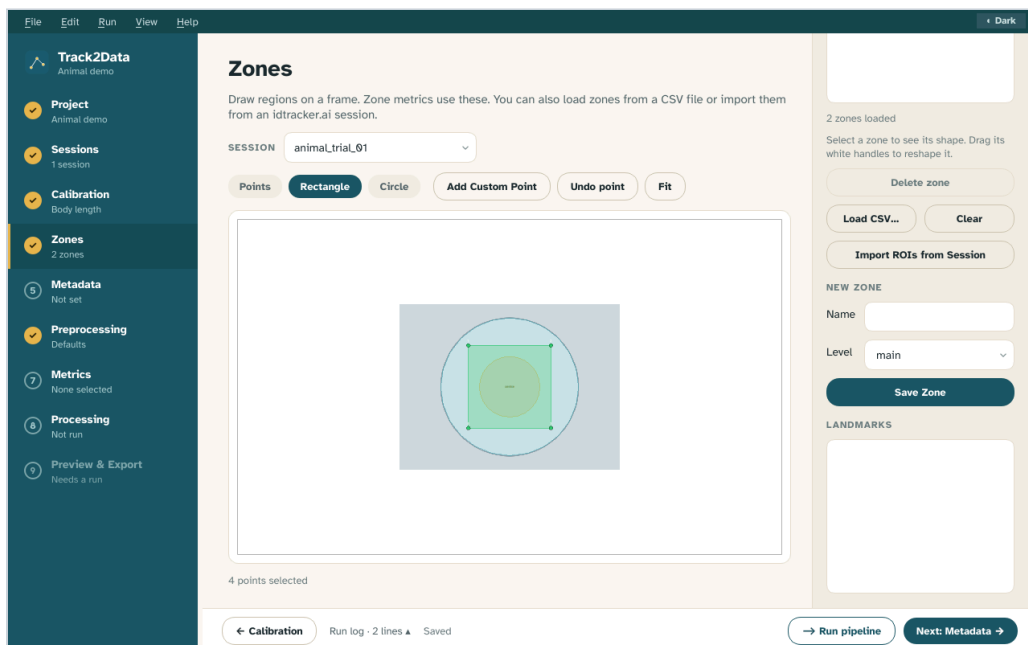


Figure 8. The zone canvas: saved zones are shaded and named, validator landmarks are blue.

Tool	How
Points	Click validator landmarks (blue) in order. Add Custom Point lets you click anywhere.
Rectangle	Drag from one corner to the opposite corner
Circle	Drag from the centre outwards
Undo point / Ctrl+Z	Remove the last vertex
Fit	Fit the whole image in view. The mouse wheel zooms; hold the middle button and drag to pan.

Custom vertices (orange/green) can be dragged to fine-tune the outline. When the shape looks right (at least 3 points), give it a **name** and a **level** (main or secondary) and press **Save Zone**.

To reshape a saved zone, select it in the list and drag its vertices. A move that would leave the zone with no area (all vertices in a line), repeat a point, or make its edges cross is refused: a message says why, the handle jumps back and the zone stays as it was.

If a zone's source resolution differs from a session's video, a yellow warning appears.

5. Metadata (optional)

Attach experimental information (treatment, date, tank, ...) to every row of the output.

Figure 9. The Metadata screen: CSV preview, column mapping and the match summary.

1. **Load metadata CSV....** The first rows are previewed.
2. Match columns to fields. Names like `date`, `condition` or `tank` are matched automatically to *Trial date*, *Treatment*, *Group ID*; change any you disagree with, or leave a field on `(skip)`.

3. The line under the mapping tells you how many sessions found a row (1 of 1 sessions matched) and names any that did not.

The `session_id` column must equal the session folder name. Without an *Individual ID* column one row is matched per session (if several rows match, the first is used and the summary says so).

Per-animal information (sex, weight, genotype of individual animals)

Give the CSV **one row per animal** and map its animal column (`fish_id`, `animal_id` or any name) to *Individual ID*. Then:

- **Match animals by** decides how each value is matched to an animal. *Validator label* uses the names set in the idtracker.ai Validator (the default labels are 1, 2, ...) and falls back to the 0-based position for a session that has none. *Position* always uses the 0-based position (the `individual_id` of the exported tables).
- Tick the other columns you want under **Also include these columns**. Columns that are not mapped to a field and not ticked are dropped.
- Per-animal values land on every row that has an `individual_id` (the per-frame table and the individual metric tables). Values that are the same for all animals of a session (such as `treatment`) also go onto the group tables; per-animal values never do.
- The summary lists animals with no row (their cells stay empty), and CSV rows whose animal matches nothing.
- **Identity-free sessions** get no per-animal values: their rows are detection slots, not animals. Session-level fields still apply.

Pick the right match mode: if your CSV counts animals from 1 and the Validator labels are the default 1, 2, ... use *Validator label*; if it counts from 0, use *Position*. A column named like an engine column (`speed_px_s`, `frame`, `bin_index`, ...) is never carried, so it cannot overwrite data.

Skip metadata removes the file if you change your mind.

6. Preprocessing

Cleaning applied to the trajectories before any metric is computed. Defaults suit most data. Every step has an **Enabled** box; the raw data is never overwritten.

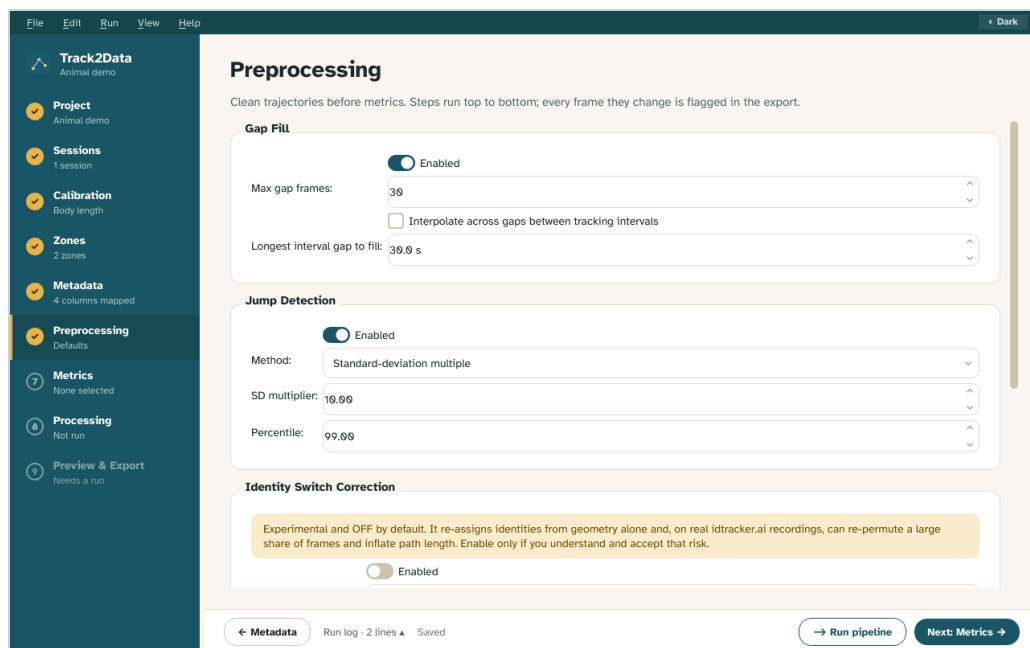


Figure 10. The Preprocessing screen with the default settings.

Step	What it does
Gap fill	Linearly interpolates tracking gaps up to <i>Max gap frames</i> (gaps between tracking intervals have their own option, below)
Jump detection	Finds implausible position jumps (standard-deviation multiple, percentile, or idtracker.ai's own velocity threshold) and replaces them
Identity switch correction	See below. Off by default
Smoothing	Moving average or Savitzky-Golay, over <i>Window</i> frames
Coverage gate	Warns when more than <i>Max NaN fraction</i> of an animal's frames are missing

Sessions tracked in separate intervals

idtracker.ai can track only some stretches of a video. Its trajectory file then holds just those frames, and Track2Data puts them back on the real timeline: speeds, smoothing and zone times are computed over the true elapsed time, and the stretches in between are **never read as adjacent frames** (that used to turn a move across a 40-second gap into one frame of motion).

By default the untracked stretch stays empty: nothing is interpolated, distance is not counted across it and a stay in a zone is cut there. Tick **Interpolate across gaps between tracking intervals** to fill gaps of up to *Longest interval gap to fill* seconds (30 s by default) with a straight line between each animal's last and next observed position. This is an assumption about where the animal went, not a measurement:

- Filled frames appear in the per-frame table with `was_interpolated` true and `in_tracking_interval` false, and are counted by the *Interpolated* column of the quality grid (diagnostic D-11).
- Distance, speed, activity and zone time include them; zone events on them carry `estimated`.

- It needs stable identities and an observed position for that animal on both sides, and never extrapolates. Sessions marked identity-free are never bridged.
- A gap longer than the limit stays a break. The limit exists so one long gap cannot quietly be replaced by a made-up path.

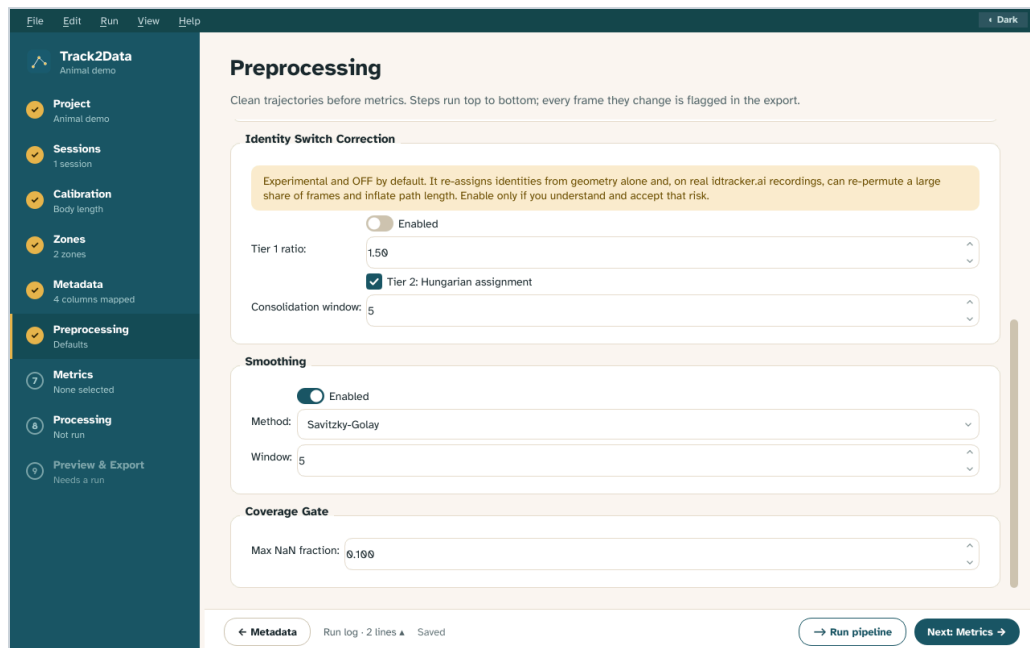


Figure 11. The identity switch correction options (experimental, off by default).

Identity switch correction is experimental and off by default. It reassigns identities from geometry alone, ignoring idtracker.ai's own fragment boundaries. On a real recording it re-permuted a large share of frames and inflated path length enormously. Leave it off unless you have checked the result in [Preview ► Trajectories](#) with *Raw + processed* shown.

Settings you change here are saved automatically and used for the next run.

7. Metrics

Choose what to compute. Metrics are grouped on three tabs: **Individual** (per animal), **Group** (across animals) and **Zone** (needs zones; the tab is disabled without them).

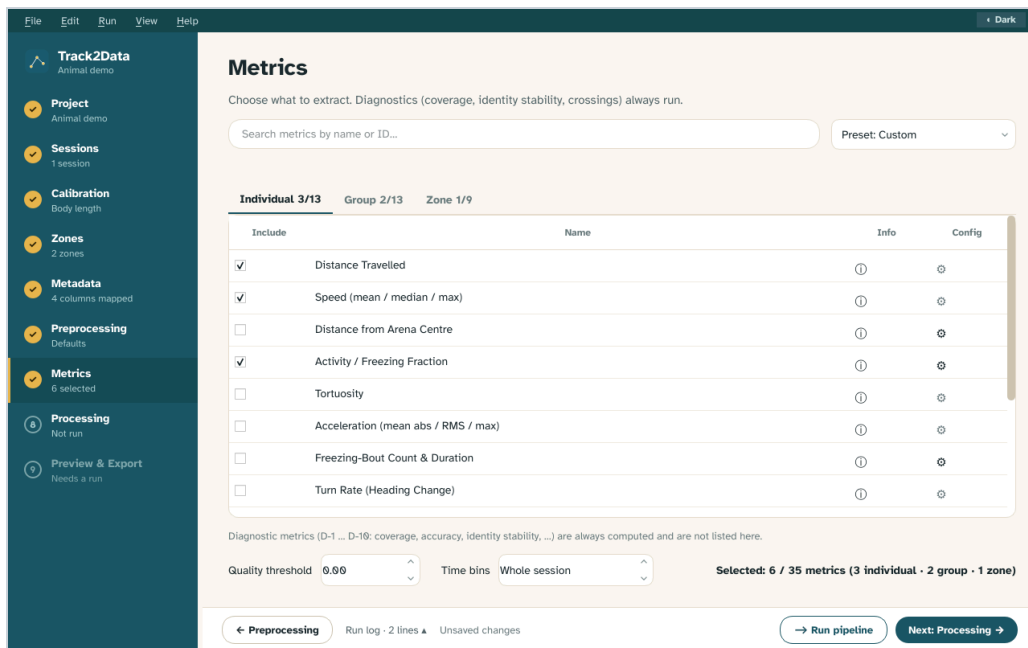


Figure 12. The Metrics screen, with search, presets and the three metric tabs.

- **Search** filters all tabs by name or ID (speed , IL-2). The line beneath says which tabs match.
- **Presets** replace the current selection: *Standard locomotor*, *Thigmotaxis & space use*, *Social dynamics*, *All metrics*.
- ⓘ opens the definition, formula and literature reference of a metric; ⚙️ edits its parameters (available on metrics that have any).
- The counter shows how many are selected.
- **Quality threshold** drops frames whose identification probability is below the value.
- **Time bins** splits every session into bins of the chosen length (minutes) and reports each metric per bin, with `bin_index`, `bin_start_s` and `bin_end_s` columns. *Whole session* (the default) turns it off. Whole-track metrics (tortuosity, home-base stability) stay one row per animal with empty bin columns; diagnostics are always whole-session. Thresholds such as the freezing speed threshold are fixed from the whole session, so bins are comparable.
- Rows are greyed out for sessions without stable identities; see [Sessions](#).

Diagnostics (coverage, tracking accuracy, identity stability, ...) are always computed and are not listed here; they appear in [Preview ► Diagnostics](#).

Every metric is documented in [docs/METRICS_SPEC.md](#).

8. Processing

Validate pipeline checks the project and lists anything that would stop a run. **Run pipeline** (or the button in the footer bar, or Ctrl+R) processes every session: import, preprocessing, calibration, zone assignment, metrics, export.

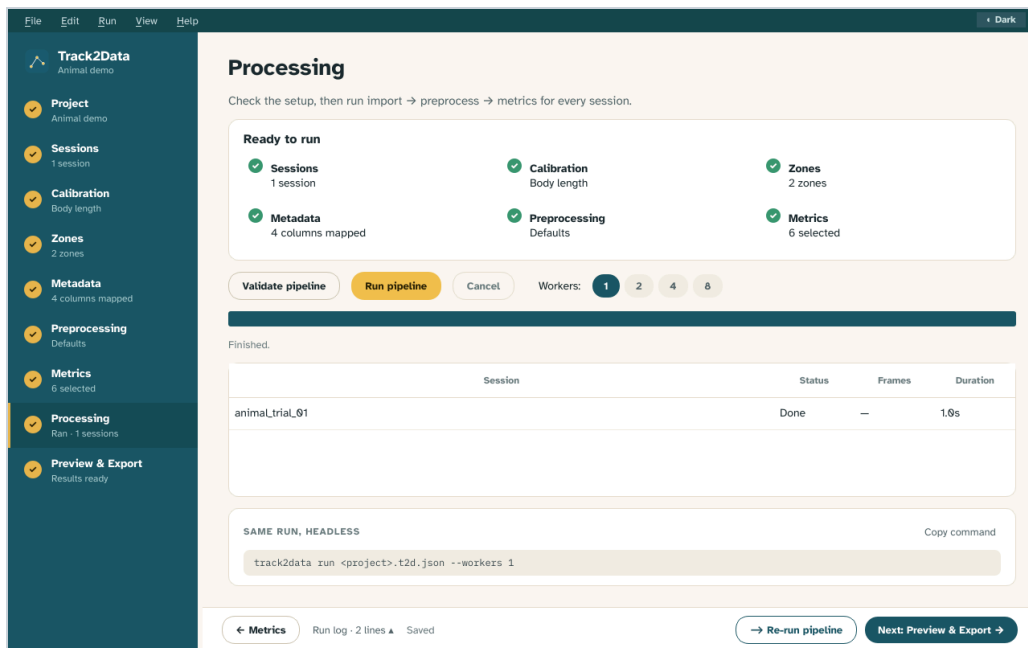


Figure 13. The Processing screen: one row per session, with status and duration.

- The table shows each session's status and duration. A failed session is marked *Failed* and does not stop the others.
- **Workers** sets how many sessions run at the same time. **1** runs them one after another. More workers help when each session takes many seconds or more; for short sessions the start-up cost of each worker makes it slower.
- **Cancel** stops the run at the next safe point (between sessions, preprocessing steps or metrics).
- Preprocessed sessions are cached, so re-running after changing only metrics or export settings skips the slow parts.

Results are written to `<project>/exports/<timestamp>/<session>/`.

9. Preview

Look before you export. Four tabs: **Summary**, **Diagnostics**, **Metrics**, **Trajectories**.

Are these results current? If you change a setting that affects the numbers (sessions, calibration, zones, metadata, preprocessing, metrics) after a run, a warning appears at the top of Preview and the sidebar says *Settings changed · re-run*: what you are looking at came from the older settings. Changing only where or in which format to export does not do this, and putting a setting back clears the warning. A run that finishes after you edited is judged against the settings it started with, so it cannot make newer edits look current, and a run that finishes for a project you have since replaced is discarded. A run in which no session succeeded is shown as *No successful sessions*, not as ready. Export recomputes the metrics under the current settings either way.

Trajectories

Choose a session and **Load trajectories** (it reuses the cache after a run). Then:

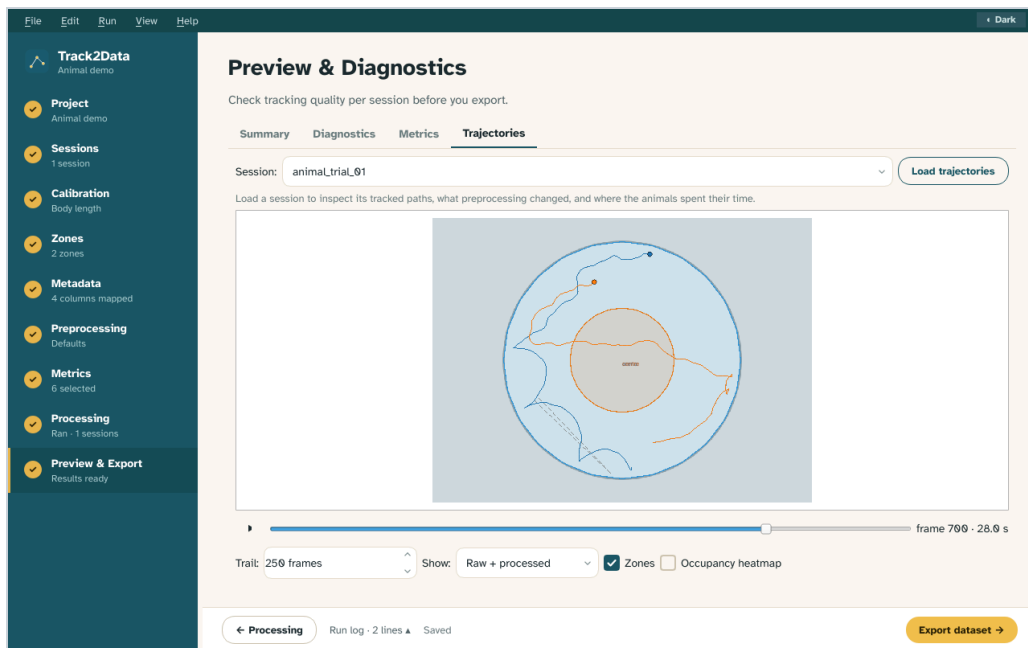


Figure 14. Trajectories with Raw + processed shown: the dashed grey line is a tracking jump that was removed.

- Drag the slider, or press ►, to move through time. **Trail** sets how many past frames are drawn.
- **Show ► Raw + processed** draws the raw path as a dashed grey line next to the processed one, so you can see exactly what gap filling, jump removal and smoothing changed. In Figure 14 the dashed line is a tracking jump that was removed.
- **Zones** shows or hides the saved zones. The mouse wheel zooms.

What to look for: a path that suddenly jumps across the arena (identity swap or detection error); long straight lines where an animal was lost; smoothing that cuts corners of real turns.

Occupancy heatmap

Tick **Occupancy heatmap** to see where the animals spent their time (red = most).

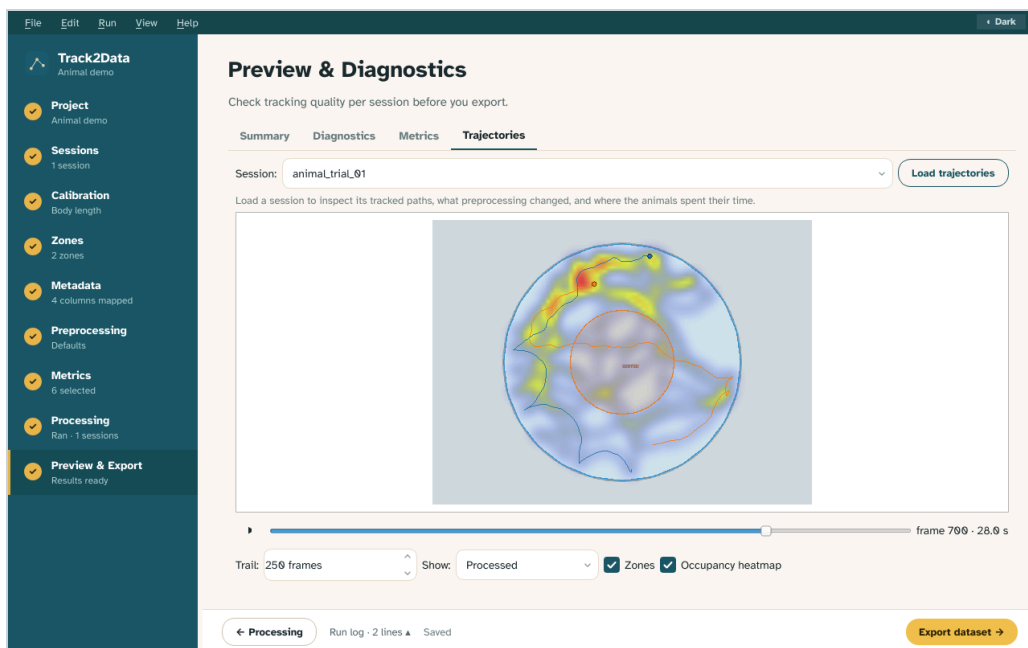


Figure 15. The occupancy heatmap.

Diagnostics

Per-animal tracking coverage and identification probability, session-level accuracy, inconsistent frames and identity stability, plus the list of preprocessing steps and how many frames each one changed.

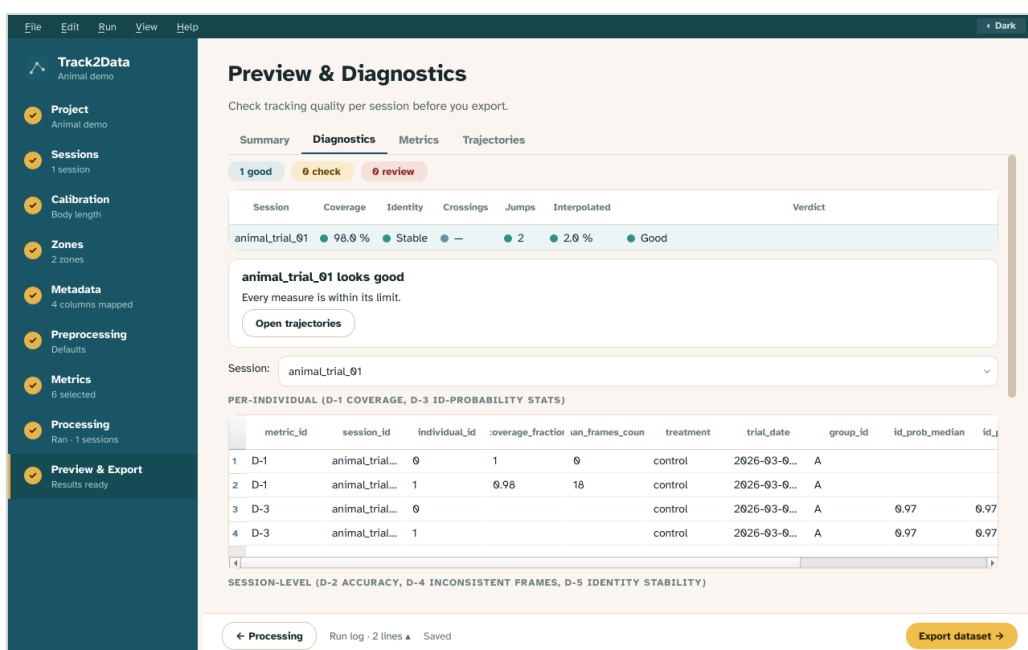


Figure 16. The Diagnostics tab.

Metrics

A table preview of each selected metric for the chosen session.

10. Export

Tick the formats you want, choose where to write them, and press **Export**.

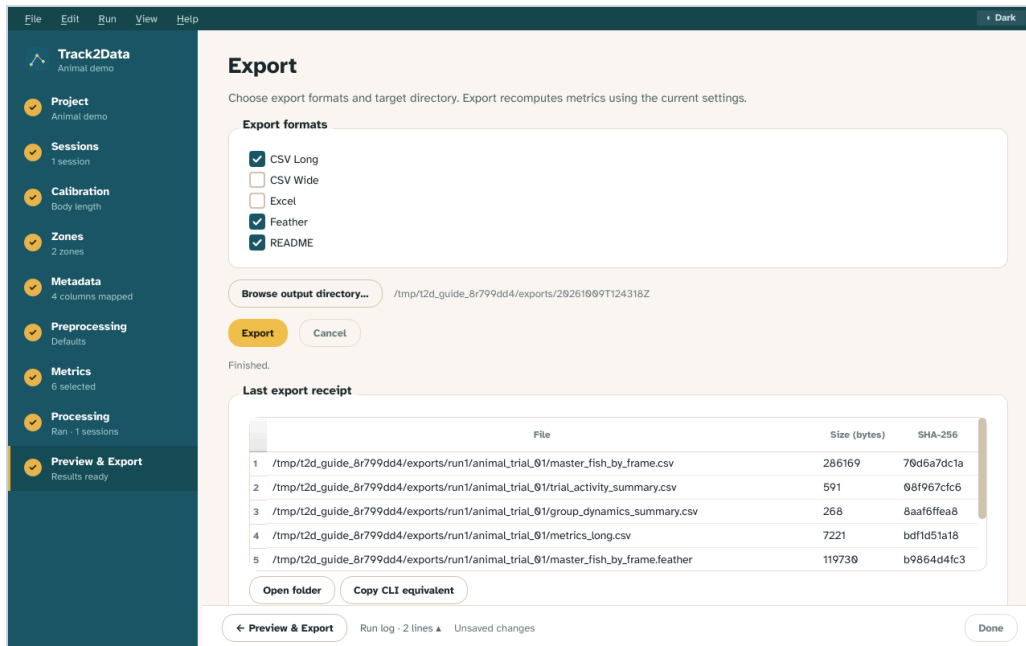


Figure 17. The Export screen, with the file receipt after an export.

1. Tick the formats you want.

Format	Files
CSV Long	master_fish_by_frame.csv, trial_activity_summary.csv, group_dynamics_summary.csv
CSV Wide	trial_summary_wide.csv
Excel	Track2Data_<project>.xlsx (one sheet per table)
Feather	master_fish_by_frame.feather, trial_activity_summary.feather
README	provenance record (always written alongside the others)

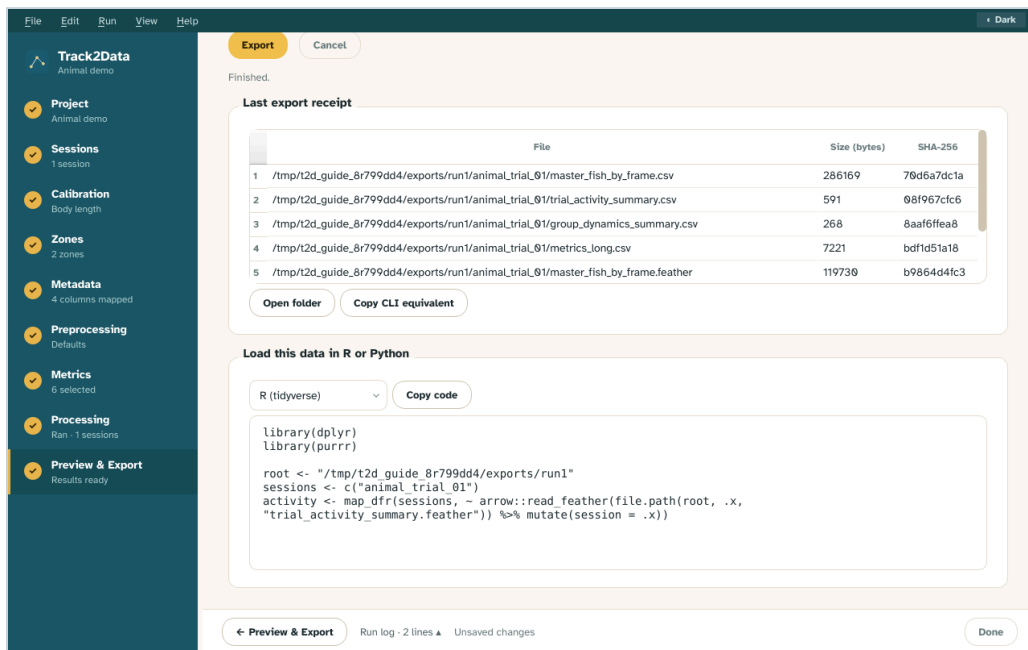
2. **Browse output directory...** or keep the default.
3. **Export.** Metrics and files are produced again under the current settings, but preprocessing comes from the cache when nothing relevant changed.

The **receipt** lists every file with its size and SHA-256. **Copy CLI equivalent** gives the `track2data run ...` command that reproduces this export.

Very long recordings: Excel limits a sheet to about one million rows, so the per-frame table continues on *Fish by Frame 2*, ... The CSV and Feather files always hold the whole table.

Load the data in R or Python

After an export, the panel at the bottom gives ready-to-paste code for the files just written (R/tidyverse or Python/pandas). **Copy code**, paste, run.



The screenshot shows the Track2Data application interface. On the left is a sidebar with a list of steps: Project, Sessions, Calibration, Zones, Metadata, Preprocessing, Metrics, Processing, and Preview & Export. The 'Preview & Export' step is currently selected. The main panel shows a 'Finished.' status and a 'Last export receipt' table. Below the table are buttons for 'Open folder' and 'Copy CLI equivalent'. At the bottom, there is a section 'Load this data in R or Python' with a dropdown menu set to 'R (tidyverse)' and a 'Copy code' button. Below this is a text area containing R code for loading the data using dplyr and purrr.

	File	Size (bytes)	SHA-256
1	/tmp/t2d_guide_8r799dd4/exports/run1/animal_trial_01/master_fish_by_frame.csv	286169	70d6a7dc1a
2	/tmp/t2d_guide_8r799dd4/exports/run1/animal_trial_01/trial_activity_summary.csv	591	08f967cfc6
3	/tmp/t2d_guide_8r799dd4/exports/run1/animal_trial_01/group_dynamics_summary.csv	268	8aaf6ffea8
4	/tmp/t2d_guide_8r799dd4/exports/run1/animal_trial_01/metrics_long.csv	7221	bdf1d51a18
5	/tmp/t2d_guide_8r799dd4/exports/run1/animal_trial_01/master_fish_by_frame.feather	119730	b9864d4fc3

```
library(dplyr)
library(purrr)

root <- "/tmp/t2d_guide_8r799dd4/exports/run1"
sessions <- c("animal_trial_01")
activity <- map_dfr(sessions, ~ arrow::read_feather(file.path(root, .x,
"trial_activity_summary.feather"))) %>% mutate(session = .x)
```

Figure 18. Ready-to-paste R and Python snippets for the files just written.

See [Understanding the output files](#) for what the columns mean.

Part II: Reference

Understanding the output files

Each session is written to `<out_dir>/<session_id>/`. Files differ by exporter; the tables are the same.

When the export holds two or more sessions, an extra folder `<out_dir>/all_sessions/` stacks the sessions together: its `master_fish_by_frame.csv`, `trial_activity_summary.csv`, `group_dynamics_summary.csv` and `metrics_long.csv` are the per-session files of the same name joined one after another, with `session_id` saying which session each row came from. Nothing is recomputed. A column that one session lacks (for example because its metric selection differed) is left blank for that session's rows. It also gets a `manifest.json` and `README.md` listing the sessions and their input checksums. Stacking is not the same as pooling: check `PROJECT_SUMMARY.md` and `sessions.csv` for differences in frame rate, group size or calibration first, and treat `session_id` as a grouping factor in your model. A single-session export has no `all_sessions/` folder. Use either the session folders or `all_sessions/`, not both: loading both counts every row twice.

Table	One row per	Contents
<code>master_fish_by_frame</code>	session × animal × frame	<code>time_s</code> , <code>x_px</code> , <code>y_px</code> , <code>was_interpolated</code> (true where preprocessing filled a gap), <code>speed_px_s</code> , <code>heading_rad</code> , <code>main_zone</code> , <code>sec_zone</code> , plus calibrated and metadata columns
<code>trial_activity_summary</code>	session × animal	individual-level metrics, plus metadata
<code>group_dynamics_summary</code>	session	group-level metrics, plus metadata
<code>trial_summary_wide</code>	session × animal	all summary metrics side by side

- **Time bins** (when set on the Metrics screen): summary tables have one row per animal *per bin*, with `bin_index`, `bin_start_s`, `bin_end_s`; `master_fish_by_frame` gets `bin_index`. Metrics that are not meaningful per window keep a single row with empty bin columns.

Column naming

Suffix	Meaning
<code>_px</code>	pixels
<code>_cm</code>	centimetres (only with a scale; otherwise empty)
<code>_bl</code>	body lengths (needs the tracker's body length; independent of the calibration mode and of any physical scale)
<code>_s</code>	per second

Things to know

- **Empty *_cm columns** mean no pixels-per-unit scale was set, not an error. Use *_bl , or set a scale on the [Calibration](#) screen.
- **Identity-free sessions** have no per-animal rows for metrics that follow an individual; zone occupancy (Z-1, Z-2, Z-8) is reported pooled over animals, without an `individual_id` column.
- **Missing positions** are empty values, never zeros.
- The README written next to the files records the settings and versions used, and the SHA-256 of each output, so a result can be traced and reproduced.

Definitions, formulas and references for every metric: [docs/METRICS_SPEC.md](#) .

Troubleshooting

You see	Why	What to do
Next is disabled	A required stage is empty or invalid	Hover over Next: the tooltip names the problem. Check the $\times$ / $\circ$ badges in the sidebar
<i>"Session calibration selected but these sessions have no length_unit"</i>	A session has no scale (idtracker.ai never calibrated it, or the other tracker's file records none)	Calibrate it in the idtracker.ai Validator, give Scale (pixels per cm) when importing, or switch the calibration mode
<i>"No reader recognised the session folder"</i>	The folder is not an idtracker.ai output	Pick the session folder itself (the one containing <code>trajectories/</code>). Supported: idtracker.ai 6.x output (the legacy v5 layout also works); v4 is not supported yet (a v4-looking folder gets a specific message; see sending a v4 sample)
Video column says <i>Not found</i>	The video path idtracker.ai recorded does not exist on this computer	Select the session and press Locate Video... (Sessions). Metrics do not need the video
<i>READER_NOT_AVAILABLE</i>	A project session was added with software this version of Track2Data does not have	Install a version that has that reader. Track2Data will not read it with a different one, since that could change the numbers
<i>READER_OPTION_MISSING</i>	The software does not record an option (such as frame rate) and none was entered	Add the folder again and fill in the <i>required</i> option in the confirm dialog
Session frames / animals show 	The folder is still being read	Wait a moment; a failed read is reported in the Run Log

You see	Why	What to do
*_cm columns are empty	No pixels-per-unit scale	Use *_bl columns or set a scale in Calibration
⚠ on <i>Sessions</i>	A session is identity-free	Expected for sessions tracked without identities; see Sessions
Individual metrics missing for a session	That session is identity-free	Untick <i>Identity-free</i> only if identities really are stable
"N of M sessions matched" with names listed	Those session_id values are not in the metadata CSV	Fix the CSV or the session folder names
A zone warning about resolution	Zones were drawn on a different image size	Redraw them on this session, or confirm the sizes really match
Path length looks far too large	Jumps or identity swaps	Open Preview ▶ Trajectories with <i>Raw + processed</i> ; keep <i>Identity switch correction</i> off
Excel file has <i>Fish by Frame 2</i>	The table exceeded Excel's row limit	Use the CSV or Feather file for the complete table
Parallel run slower than sequential	Short sessions; each worker takes time to start	Use 1 worker
macOS says the app is damaged / Windows SmartScreen blocks it	Releases are not signed yet	See docs/CODE_SIGNING.md

Still stuck? Open the **Run Log** (View ▶ Toggle Run Log), then an issue on GitHub with its text.